

Cours de Computer Vision

Fondements, modèles modernes et démarche pratique

Synthèse de cours

2026

Table des matières

Mode d'emploi	3
1 Images numériques et formulation des tâches	4
1.1 Pourquoi la vision est difficile	4
1.2 Tâches principales	4
1.3 Données et split	4
2 Filtres, caractéristiques et convolution	5
2.1 Dimensions	5
2.2 Approche classique et approche apprise	5
3 Réseaux de neurones et apprentissage	6
3.1 Losses	6
3.2 Optimisation	6
4 Réseaux convolutifs	7
4.1 Architecture type	7
4.2 Variantes efficaces	7
5 Architectures CNN et Transfer Learning	8
5.1 ResNet	8
5.2 Transfer Learning	8
6 Classification d'images	9
6.1 Déséquilibre	9
6.2 Analyse d'erreurs	9
7 Détection d'objets	10
7.1 Intersection over Union	10
7.2 Deux familles	10
7.3 Non-Maximum Suppression	10
8 YOLO	11

8.1	Sorties et loss	11
8.2	Pipeline	11
9	Segmentation d'images	12
9.1	Encodeur-décodeur	12
9.2	Losses et métriques	12
10	Vision Transformers et modèles vision-langage	13
10.1	DETR et vision-langage	13
11	Données, annotations et augmentation	14
11.1	Augmentations	14
12	Métriques et évaluation	15
12.1	Détection	15
12.2	Segmentation et calibration	15
13	Robustesse, explicabilité et déploiement	16
13.1	Explicabilité	16
13.2	Production	16
13.3	Choisir une approche face à un problème réel	16
13.4	Protocole final	16
13.5	Conclusion	17
A	Aide-mémoire mathématique	18

Mode d'emploi

Ce cours suit une progression allant des pixels à la mise en production. Les encadrés *Réflexe* résument les décisions pratiques ; les encadrés *Attention* signalent les pièges fréquents. Les modèles récents sont présentés comme des prolongements des principes fondamentaux, et non comme un catalogue d'architectures.

Chapitre 1

Images numériques et formulation des tâches

La vision par ordinateur transforme des mesures visuelles en informations utiles. Une image en niveaux de gris est une matrice $X \in \mathbb{R}^{H \times W}$; une image couleur est un tenseur $X \in \mathbb{R}^{H \times W \times C}$, avec généralement $C = 3$ pour RGB. Un batch PyTorch suit souvent la convention $N \times C \times H \times W$.

1.1 Pourquoi la vision est difficile

L'apparence varie avec l'éclairage, le point de vue, l'échelle, l'occultation, le capteur et l'arrière-plan. Il existe une variabilité intra-classe et une similarité inter-classe. Avant toute modélisation, il faut vérifier que l'information nécessaire est réellement visible.

1.2 Tâches principales

Tâche	Sortie	Annotation
Classification	Une classe globale	Label image
Multilabel	Plusieurs catégories	Vecteur binaire
Détection	Classes et boîtes	Bounding boxes
Segmentation sémantique	Classe par pixel	Masque
Segmentation d'instances	Masque par objet	Masques d'instances
Pose	Points caractéristiques	Keypoints
Profondeur	Distance par pixel	Carte de profondeur

Réflexe. Choisir la sortie minimale permettant la décision métier. Si une classe suffit, ne pas payer le coût d'annotation d'une segmentation.

1.3 Données et split

Le dataset doit représenter les conditions de production. Les images corrélées doivent rester dans le même ensemble : patient, vidéo, caméra, site ou objet physique. Le split est construit avant l'augmentation afin d'éviter qu'une variante de la même image apparaisse dans Train et Test.

Chapitre 2

Filtres, caractéristiques et convolution

Un filtre spatial applique un noyau local à chaque position. Pour un noyau K et une image X , l'opération utilisée dans les bibliothèques de deep learning est généralement une corrélation croisée :

$$Y_{i,j} = \sum_u \sum_v K_{u,v} X_{i+u,j+v}.$$

La sortie est une *feature map*. Les filtres classiques incluent le flou gaussien, la netteté et Sobel. Les gradients horizontaux et verticaux donnent

$$\|\nabla X\| = \sqrt{G_x^2 + G_y^2}, \quad \theta = \text{atan2}(G_y, G_x).$$

2.1 Dimensions

Pour une taille N , un kernel K , un padding P , un stride S et une dilation D :

$$N_{\text{out}} = \left\lfloor \frac{N + 2P - D(K - 1) - 1}{S} \right\rfloor + 1.$$

Le partage des poids et la connectivité locale réduisent fortement le nombre de paramètres. Une couche avec C_{in} canaux, C_{out} filtres et des kernels $K_h \times K_w$ possède

$$K_h K_w C_{in} C_{out} + C_{out}$$

paramètres lorsque les biais sont utilisés.

2.2 Approche classique et approche apprise

La vision classique conçoit des descripteurs (HOG, SIFT, histogrammes) puis entraîne un SVM ou un autre modèle. Un CNN apprend simultanément les filtres et la décision finale. Une méthode classique reste pertinente dans un environnement contrôlé lorsque la règle visuelle est simple.

Chapitre 3

Réseaux de neurones et apprentissage

Un neurone calcule

$$z = \mathbf{w}^\top \mathbf{x} + b, \quad a = f(z).$$

Sans activation non linéaire, une composition de couches affines reste affine. ReLU, $\max(0, z)$, est une activation classique ; sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

sert notamment aux sorties binaires et multilabel. Softmax transforme K logits en probabilités exclusives :

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}.$$

3.1 Losses

Pour le binaire :

$$\mathcal{L}_{BCE} = -\frac{1}{N} \sum_i [y_i \log p_i + (1 - y_i) \log(1 - p_i)].$$

Pour le multiclasse :

$$\mathcal{L}_{CE} = -\frac{1}{N} \sum_i \log p_{i, y_i}.$$

Les implémentations stables reçoivent directement les logits : ne pas appliquer sigmoid avant `BCEWithLogitsLoss`, ni softmax avant `CrossEntropyLoss`.

3.2 Optimisation

La rétropropagation calcule $\nabla_\theta \mathcal{L}$ par la règle de chaîne. La descente de gradient met à jour

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t).$$

SGD avec Momentum et AdamW sont des optimiseurs usuels. Le learning rate est central. Une epoch parcourt le Train ; une itération traite un mini-batch. Suivre séparément les courbes Train et Validation permet de détecter underfitting et overfitting.

Réflexe. Avant un entraînement long, vérifier que le modèle peut mémoriser un très petit batch. Sinon, chercher une erreur de données, dimensions, loss ou gradients.

Chapitre 4

Réseaux convolutifs

Une couche reçoit $X \in \mathbb{R}^{N \times C_{in} \times H \times W}$ et produit $Y \in \mathbb{R}^{N \times C_{out} \times H' \times W'}$. Les premières couches apprennent souvent bords et couleurs; les couches profondes combinent textures, parties et concepts.

4.1 Architecture type

```
Image -> [Conv -> BatchNorm -> ReLU] x n
        -> reduction spatiale
        -> Global Average Pooling
        -> couche lineaire -> logits
```

Le champ réceptif croît avec la profondeur. Pour un kernel K_l , un stride S_l , un saut j_l et un champ r_l :

$$j_l = j_{l-1}S_l, \quad r_l = r_{l-1} + (K_l - 1)j_{l-1}.$$

Le pooling réduit la résolution; le Global Average Pooling calcule

$$g_c = \frac{1}{HW} \sum_i \sum_j A_{c,i,j}.$$

4.2 Variantes efficaces

Une convolution 1×1 combine les canaux. Une convolution dilatée possède une taille effective $K_{eff} = D(K - 1) + 1$. Une convolution depthwise separable réduit le coût de $K^2 C_{in} C_{out}$ à environ $K^2 C_{in} + C_{in} C_{out}$ par position.

Chapitre 5

Architectures CNN et Transfer Learning

Les architectures doivent être retenues par leur idée principale : LeNet établit le pipeline ; AlexNet démontre l'entraînement profond sur GPU ; VGG empile des kernels 3×3 ; Inception traite plusieurs échelles ; MobileNet emploie des convolutions séparables ; EfficientNet équilibre profondeur, largeur et résolution ; ConvNeXt modernise le CNN.

5.1 ResNet

Un bloc résiduel apprend

$$\mathbf{y} = F(\mathbf{x}) + \mathbf{x}.$$

Le gradient contient un chemin identité :

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \frac{\partial F}{\partial \mathbf{x}} + I.$$

Si les dimensions changent, une projection donne $\mathbf{y} = F(\mathbf{x}) + W_s \mathbf{x}$. DenseNet concatène plutôt les sorties antérieures.

5.2 Transfer Learning

Un backbone pré-entraîné fournit une représentation $\mathbf{h}$; une nouvelle tête produit $\mathbf{z} = W\mathbf{h} + \mathbf{b}$. Stratégie recommandée : remplacer la tête, geler le backbone, entraîner la tête, puis dégeler progressivement les derniers blocs avec $\eta_{backbone} < \eta_{head}$. Le prétraitement doit correspondre aux poids pré-entraînés.

Attention. Geler les gradients n'est pas identique au mode évaluation : BatchNorm et Dropout doivent être contrôlés explicitement.

Chapitre 6

Classification d'images

En binaire, utiliser un logit et BCE. En multiclasse exclusif, utiliser K logits et Cross-Entropy. En multilabel, utiliser K sigmoids indépendantes et BCE. Pour le multiclasse :

$$\hat{y} = \arg \max_k z_k.$$

6.1 Déséquilibre

Une Cross-Entropy pondérée s'écrit

$$\mathcal{L} = -\frac{1}{N} \sum_i w_{y_i} \log p_{i,y_i}.$$

La Focal Loss concentre l'apprentissage sur les exemples difficiles :

$$\mathcal{L}_{focal} = -\alpha_t (1 - p_t)^\gamma \log p_t.$$

Évaluer avec matrice de confusion, precision, recall et Macro F1, pas seulement accuracy.

6.2 Analyse d'erreurs

Inspecter les erreurs fortement confiantes, les classes rares et les sous-groupes. Rechercher les raccourcis visuels : arrière-plan, filigrane ou capteur corrélé au label. Grad-CAM peut localiser les régions influentes, sans constituer une preuve causale.

Chapitre 7

Détection d'objets

La détection prédit pour chaque objet une classe, une boîte et un score. Une boîte peut être représentée par $(x_{min}, y_{min}, x_{max}, y_{max})$ ou (x_c, y_c, w, h) .

7.1 Intersection over Union

$$IoU(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

Une détection est associée à un objet réel selon un seuil d'IoU, puis classée TP, FP ou FN. Les objets manqués sont des FN ; doublons et mauvaises localisations deviennent souvent des FP.

7.2 Deux familles

Les détecteurs à deux étapes (Faster R-CNN) proposent des régions puis les classent : bonne précision, coût supérieur. Les détecteurs à une étape prédisent directement sur des feature maps : meilleure vitesse. Les Feature Pyramid Networks combinent plusieurs résolutions pour traiter les objets de tailles différentes.

7.3 Non-Maximum Suppression

NMS trie les boîtes par score, conserve la meilleure et supprime les boîtes très chevauchantes de même classe. Un seuil trop faible supprime des objets proches ; trop élevé conserve des doublons.

Chapitre 8

YOLO

YOLO est une famille de détecteurs one-stage orientés vers un bon compromis vitesse-précision. Un backbone extrait les features, un neck fusionne les échelles, puis une head prédit classes, objectness et géométrie.

8.1 Sorties et loss

Une loss schématique est

$$\mathcal{L} = \lambda_{box}\mathcal{L}_{box} + \lambda_{obj}\mathcal{L}_{obj} + \lambda_{cls}\mathcal{L}_{cls}.$$

Les versions modernes peuvent être anchor-based ou anchor-free. Il faut retenir les principes plutôt qu'un numéro de version : prédiction multi-échelle, attribution des cibles, loss de boîte et post-traitement.

8.2 Pipeline

```
Image -> resize/letterbox -> reseau -> boites brutes  
      -> seuil de confiance -> NMS -> detections finales
```

Pour déployer, mesurer mAP, recall, latence, débit, mémoire et performance par taille d'objet. Une résolution plus grande aide les petits objets mais augmente le coût.

Chapitre 9

Segmentation d'images

La segmentation sémantique prédit une classe par pixel ; la segmentation d'instances distingue les objets ; la segmentation panoptique combine *stuff* et *things*.

9.1 Encodeur-décodeur

L'encodeur réduit la résolution et construit le contexte. Le décodeur reconstruit une carte dense. U-Net transmet des features haute résolution par skip connections :

$$F_{dec}^{(l)} = \text{Concat}(\text{Up}(F_{dec}^{(l+1)}), F_{enc}^{(l)}).$$

DeepLab emploie notamment des convolutions dilatées et un contexte multi-échelle. Mask R-CNN ajoute une tête de masque à un détecteur d'instances.

9.2 Losses et métriques

Pour une classe :

$$Dice = \frac{2|P \cap G|}{|P| + |G|}, \quad IoU = \frac{|P \cap G|}{|P \cup G|}.$$

Une version lissée de la Dice Loss est

$$\mathcal{L}_{Dice} = 1 - \frac{2 \sum_i p_i g_i + \varepsilon}{\sum_i p_i + \sum_i g_i + \varepsilon}.$$

La combinaison Cross-Entropy + Dice est fréquente lorsque le fond domine.

Chapitre 10

Vision Transformers et modèles vision-langage

Une image est découpée en patches. Chaque patch aplati est projeté :

$$\mathbf{e}_i = W_p \mathbf{x}_i + \mathbf{p}_i,$$

où $\mathbf{p}_i$ encode la position. L'attention est

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V.$$

Le coût quadratique en nombre de tokens motive fenêtres locales et architectures hiérarchiques.

10.1 DETR et vision-langage

DETR formule la détection comme une prédiction d'ensemble avec appariement biparti. CLIP rapproche représentations image et texte dans un espace commun, permettant notamment du zero-shot. Les modèles multimodaux peuvent décrire et interroger une image, mais restent sensibles aux hallucinations, biais et détails fins.

Complément. Les modèles de fondation ne suppriment pas le besoin d'un Test représentatif, de métriques métier et d'une validation humaine pour les usages critiques.

Chapitre 11

Données, annotations et augmentation

Une taxonomie claire définit classes, cas limites, occultations et objets ignorés. Mesurer les désaccords entre annotateurs et auditer les labels. Les splits doivent refléter les groupes et la chronologie de production.

11.1 Augmentations

Une transformation T est valide si elle conserve la cible :

$$y(T(x)) = y(x).$$

Transformations courantes : crop, flip, rotation modérée, color jitter, bruit, blur, Cutout, MixUp et CutMix. En détection et segmentation, transformer simultanément image, boîtes, masques et keypoints.

MixUp construit

$$\tilde{x} = \lambda x_i + (1 - \lambda)x_j, \quad \tilde{y} = \lambda y_i + (1 - \lambda)y_j.$$

Une augmentation trop forte peut détruire le signal ou créer des images irréalistes. Ne jamais augmenter Validation et Test avec des transformations aléatoires ordinaires.

Chapitre 12

Métriques et évaluation

En classification :

$$Precision = \frac{TP}{TP + FP}, \quad Recall = \frac{TP}{TP + FN}, \quad F1 = 2 \frac{PR}{P + R}.$$

Macro moyenne les classes également ; Weighted pondère par support ; Micro agrège les décisions.

12.1 Détection

Une courbe precision-recall est obtenue en faisant varier le seuil. L’Average Precision résume la courbe ; la mAP moyenne sur les classes et, selon le protocole, plusieurs seuils d’IoU. Toujours annoncer le protocole exact, car “mAP” seul est ambigu.

12.2 Segmentation et calibration

Rapporter IoU/Dice par classe et macro, pas seulement la pixel accuracy. Pour la calibration, comparer confiance et fréquence empirique de succès. L’ECE s’écrit

$$ECE = \sum_m \frac{|B_m|}{N} |acc(B_m) - conf(B_m)|.$$

Évaluer aussi latence, débit, mémoire, taille du modèle et consommation lorsque la production l’exige.

Chapitre 13

Robustesse, explicabilité et déploiement

Tester les changements d'éclairage, capteur, compression, flou, météo, population et domaine. Un modèle peut apprendre un raccourci corrélé au label. L'analyse d'erreurs doit croiser classe, confiance, sous-groupe et condition d'acquisition.

13.1 Explicabilité

Grad-CAM pondère les feature maps par les gradients de la classe :

$$\alpha_k^c = \frac{1}{HW} \sum_{i,j} \frac{\partial z_c}{\partial A_{ij}^k}, \quad L^c = \text{ReLU} \left(\sum_k \alpha_k^c A^k \right).$$

Une carte d'activation explique le comportement du modèle, pas une causalité dans le monde réel.

13.2 Production

Exporter avec le prétraitement, l'ordre des classes, les seuils et les versions. Optimisations possibles : quantification, pruning, distillation, compilation et batching. Surveiller dérive des entrées, confiance, taux de rejet, latence et performance lorsque les labels arrivent.

13.3 Choisir une approche face à un problème réel

Réflexe. Une classe globale : classification. **Objets et positions** : détection. **Contours précis** : segmentation. **Peu de données** : backbone pré-entraîné. **Temps réel/embarqué** : modèle léger et résolution maîtrisée. **Usage critique** : calibration, rejet, audit par sous-groupes et validation humaine.

13.4 Protocole final

1. Définir la décision métier, les erreurs coûteuses et les contraintes.
2. Choisir la tâche et le niveau minimal d'annotation.
3. Auditer labels, groupes, diversité et représentativité.

4. Construire Train, Validation et Test avant l'augmentation.
5. Créer une baseline simple et un modèle pré-entraîné.
6. Choisir loss et métriques cohérentes avec le problème.
7. Entraîner, suivre les courbes et conserver le meilleur checkpoint.
8. Évaluer par classe, sous-groupe, condition et niveau de confiance.
9. Inspecter visuellement les erreurs et rechercher les raccourcis.
10. Mesurer le coût réel : latence, mémoire, débit et énergie.
11. Déployer avec prétraitement et post-traitement versionnés.
12. Surveiller les dérives et organiser le réentraînement.

13.5 Conclusion

Le meilleur système n'est pas nécessairement le réseau le plus récent. C'est celui qui utilise une formulation correcte, des données représentatives, une évaluation honnête et une architecture compatible avec les contraintes réelles. La progression recommandée est :

<pre>Baseline simple -> modele pre-entraîne -> fine-tuning -> analyse d'erreurs -> optimisation production</pre>
--

Annexe A

Aide-mémoire mathématique

$$\begin{aligned}\sigma(z) &= \frac{1}{1 + e^{-z}}, & p_k &= \frac{e^{z_k}}{\sum_j e^{z_j}} \\ N_{out} &= \left\lfloor \frac{N + 2P - D(K - 1) - 1}{S} \right\rfloor + 1 \\ IoU &= \frac{|A \cap B|}{|A \cup B|}, & Dice &= \frac{2|A \cap B|}{|A| + |B|} \\ \theta_{t+1} &= \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t)\end{aligned}$$